

# SLOT4: Mongoose

## Tóm tắt

10.1 NoSQL Best Practices

10.2 MongoDB Mongo Shell Features

10.3 NodeJs and MongoDB

Mongoose:

11 What is Mongoose

Mongoose Population

12 Use Mongoose population to cross-reference users within a comment

## 10.1 NoSQL Best Practices

- Hiểu các nguyên tắc thiết kế cơ sở dữ liệu NoSQL hiệu quả.
- Tối ưu hóa mô hình dữ liệu NoSQL dựa trên cách sử dụng thực tế.
- Nắm rõ các kỹ thuật như phân mảnh (sharding), lập chỉ mục (indexing), và kiểm soát tính nhất quán (consistency).

## 10.2 MongoDB Mongo Shell Features

- Làm quen với các lệnh cơ bản và nâng cao trong Mongo Shell.
- Thực hành truy vấn, cập nhật, và xóa dữ liệu qua giao diện dòng lệnh.
- Sử dụng Mongo Shell để thao tác với cơ sở dữ liệu MongoDB hiệu quả.

## 10.3 Node.js and MongoDB

- Kết nối ứng dụng Node.js với MongoDB.
- Thực hiện các thao tác CRUD từ Node.js.
- Hiểu và xử lý bất đồng bộ (async/await hoặc callback) trong khi tương tác với MongoDB.

## 11. Mongoose

### What is Mongoose

- Giới thiệu về Mongoose – thư viện ODM cho MongoDB trong Node.js.
- Hiểu cách định nghĩa schema, model và sử dụng để thao tác dữ liệu.

### Mongoose Population

- Khái niệm “population” trong Mongoose để tham chiếu giữa các collection.
- Thực hành sử dụng `.populate()` để truy vấn dữ liệu liên kết.

## 12. Use Mongoose Population to Cross-reference Users within a Comment

- Xây dựng mô hình dữ liệu có liên kết giữa `User` và `Comment`.
- Sử dụng Mongoose `.populate()` để tự động truy xuất thông tin người dùng trong mỗi comment.
- Hiểu và thực hành quan hệ một-nhiều trong MongoDB bằng Mongoose

## ✓ CÁC NỘI DUNG TRỌNG TÂM, CỐT LÕI CẦN LƯU Ý:

### ◆ 10.1 – NoSQL Best Practices

#### ✓ Nội dung cốt lõi:

- Thiết kế dữ liệu dựa theo nhu cầu truy vấn (query-first design).
- Không có join phức tạp như SQL → Nên tổ chức dữ liệu gọn gàng, dễ truy xuất.
- Denormalization (phi chuẩn hóa): Sao chép dữ liệu khi cần để tăng hiệu suất.
- Dùng Index đúng cách: Tăng tốc tìm kiếm, lọc dữ liệu.
- Hiểu rõ 4 loại NoSQL chính: Document, Key-Value, Column, Graph.

📌 Lưu ý quan trọng: MongoDB là dạng Document Store – rất linh hoạt, nhưng dễ gây rối nếu không thiết kế tốt từ đầu.

### ◆ 10.2 – MongoDB Mongo Shell Features

#### ✓ Nội dung cốt lõi:

- Câu lệnh cơ bản:
  - `use <database>` – chọn DB
  - `db.collection.insertOne()` – thêm dữ liệu
  - `db.collection.find()` – tìm kiếm
  - `db.collection.updateOne()` – cập nhật
  - `db.collection.deleteOne()` – xóa
- Dữ liệu trong MongoDB là dạng JSON-like (BSON).

📌 Lưu ý quan trọng: Mongo Shell là công cụ mạnh để test và thao tác nhanh, nhưng chỉ nên dùng ở môi trường kiểm thử/trước khi triển khai thực tế.

### ◆ 10.3 – Node.js and MongoDB

#### ✓ Nội dung cốt lõi:

- Kết nối MongoDB từ Node.js sử dụng thư viện `mongodb`.
- Biết cách sử dụng các thao tác **CRUD** trong mã Node.js:
  - `insertOne`, `find`, `updateOne`, `deleteOne`
- Xử lý bất đồng bộ (asynchronous) với `async/await`.

📌 Lưu ý quan trọng: Luôn đóng kết nối sau khi thao tác xong (`client.close()`), và sử dụng `try/catch` để bắt lỗi khi thao tác DB.

### ◆ 11 – Mongoose: ODM cho MongoDB

#### ✓ Nội dung cốt lõi:

- Schema: Xác định cấu trúc của document.
- Model: Là lớp đại diện cho một collection.
- Tạo dữ liệu mới: `new Model()` + `.save()`

- **Truy vấn dữ liệu:** `Model.find()`, `Model.findById()`, v.v.

📌 **Lưu ý quan trọng:** Mongoose giúp bạn định nghĩa rõ cấu trúc và ràng buộc dữ liệu (validation, kiểu dữ liệu, mặc định, v.v.)

## 📦 Mongoose Population

### ✅ Nội dung cốt lõi:

- **Khái niệm:** `.populate()` dùng để liên kết và truy vấn dữ liệu ở collection khác.
- Ví dụ: Trong `Comment`, có trường `user: ObjectId` → dùng `.populate('user')` để truy xuất thông tin đầy đủ về người dùng đó.

### 📌 Lưu ý quan trọng:

- Cần định nghĩa đúng kiểu liên kết (`ref: 'TênModel'`) trong schema.
- Dùng `.populate()` khi bạn cần lấy dữ liệu liên quan mà không muốn lưu dư thừa.

## 📦 12 – Cross-reference Users in Comments

### ✅ Nội dung cốt lõi:

- Biết cách tạo quan hệ giữa `Comment` và `User` bằng `ObjectId`.
- Dùng `.populate()` để hiển thị thông tin người dùng trong mỗi comment.

### 📌 Lưu ý quan trọng:

- Đây là **cách làm chuẩn** để quản lý mối quan hệ một-nhiều (`User – Comments`).
- Tránh lặp lại thông tin người dùng trong nhiều comment → dễ sai lệch khi user đổi tên/email.

## 🔗 TỔNG KẾT NHANH – CẦN GHI NHỚ:

| Nội dung                 | Ghi nhớ chính                                           |
|--------------------------|---------------------------------------------------------|
| <b>NoSQL</b>             | Thiết kế dựa theo truy vấn – không join – index         |
| <b>Mongo Shell</b>       | Lệnh thao tác dữ liệu: insert, find, update, delete     |
| <b>Node.js + MongoDB</b> | Dùng <code>mongodb</code> để thực hiện CRUD bất đồng bộ |
| <b>Mongoose</b>          | Schema + Model giúp kiểm soát dữ liệu                   |
| <b>Populate</b>          | Liên kết giữa collections (VD: user trong comment)      |

### ✅ CÁC ỨNG DỤNG THỰC TẾ

#### 📁 1. NoSQL & MongoDB – Ứng dụng thực tế

### 📌 Khi nào dùng:

- Khi dữ liệu linh hoạt, không cố định (schema-free)
- Khi hệ thống cần xử lý lượng lớn dữ liệu và mở rộng dễ dàng

### Ứng dụng:

- **Ứng dụng mạng xã hội** (Facebook, Twitter): Lưu bài đăng, bình luận, lượt thích – dữ liệu đa dạng và thay đổi thường xuyên.
- **Sàn thương mại điện tử** (Shopee, Tiki): Mỗi sản phẩm có đặc tính khác nhau, dùng document store sẽ linh hoạt hơn SQL.
- **Hệ thống phân tích log**: Lưu trữ log từ nhiều hệ thống khác nhau với cấu trúc không đồng nhất.

## 2. Mongo Shell – Dành cho nhà phát triển và quản trị viên

### Khi nào dùng:

- Khi bạn cần thao tác trực tiếp với dữ liệu
- Khi cần kiểm tra nhanh, fix dữ liệu hoặc thử truy vấn

### Ứng dụng:

- Quản trị viên kiểm tra dữ liệu người dùng.
- Nhà phát triển test trực tiếp dữ liệu khi chưa có giao diện frontend.
- Debug lỗi dữ liệu nhanh chóng trong môi trường sản xuất.

## 3. Node.js + MongoDB – Xây dựng backend thực tế

### Khi nào dùng:

- Khi cần xây dựng RESTful API hoặc backend realtime
- Khi muốn tạo các ứng dụng web hiện đại (chat, blog, thương mại điện tử)

### Ứng dụng:

- **Blog cá nhân**: CRUD bài viết, bình luận, tài khoản người dùng.
- **Ứng dụng đặt đồ ăn** (giống GrabFood): Quản lý đơn hàng, nhà hàng, người dùng.
- **Ứng dụng chat**: Mỗi tin nhắn được lưu trong MongoDB, xử lý realtime với WebSocket và Node.js.

## 4. Mongoose – Quản lý dữ liệu dễ dàng hơn

### Khi nào dùng:

- Khi làm việc với MongoDB trong Node.js và muốn có kiểm soát rõ ràng hơn

## Ứng dụng:

- Đảm bảo rằng mỗi document đều tuân thủ cấu trúc nhất định (ví dụ: User phải có tên, email).
- Tự động hóa kiểm tra dữ liệu, validate dữ liệu đầu vào (email hợp lệ, mật khẩu đủ dài).
- Tái sử dụng model nhiều nơi trong ứng dụng (gọn, hiệu quả).

## 5. Mongoose Population – Xử lý mối quan hệ giữa các dữ liệu

### Khi nào dùng:

- Khi cần liên kết dữ liệu giữa các collection, giống như "join" trong SQL.

## Ứng dụng:

- **Blog hoặc diễn đàn:** Hiện thị comment kèm theo tên người dùng (dùng `.populate("user")`).
- **Hệ thống đặt hàng:** Liên kết đơn hàng với user và sản phẩm.
- **Website học trực tuyến:** Bài học liên kết với giảng viên, khóa học liên kết với học viên.

### Ví dụ:

Bạn có một trang hiển thị comment của bài viết. Thay vì lưu toàn bộ thông tin người dùng trong mỗi comment (gây dư thừa), bạn chỉ lưu `userId` và dùng `.populate('user')` để tự động truy xuất thông tin user khi cần.

## 1. Ứng Dụng Thời Gian Thực (Real-Time Applications)

**Ví dụ:** Ứng dụng chat thời gian thực

- **Công nghệ:** Node.js, Express.js, MongoDB, Mongoose, WebSockets
- **Mô tả:** Ứng dụng chat cho phép người dùng gửi và nhận tin nhắn trong thời gian thực. Mongoose được sử dụng để quản lý dữ liệu người dùng và tin nhắn, trong khi WebSockets xử lý việc truyền tải dữ liệu thời gian thực giữa server và client.

## 2. Hệ Thống Quản Lý Nội Dung (Content Management Systems - CMS)

**Ví dụ:** Hệ thống quản lý blog

- **Công nghệ:** Node.js, Express.js, MongoDB, Mongoose, EJS (Embedded JavaScript)
- **Mô tả:** Hệ thống cho phép người dùng tạo, chỉnh sửa, và xóa các bài viết blog. Mongoose quản lý dữ liệu bài viết, tác giả, và bình luận. Express.js xử lý các yêu cầu HTTP và EJS render các trang HTML.

## 3. Ứng Dụng Thương Mại Điện Tử (E-commerce Applications)

**Ví dụ:** Ứng dụng thương mại điện tử

- **Công nghệ:** Node.js, Express.js, MongoDB, Mongoose
- **Mô tả:** Ứng dụng cho phép người dùng duyệt sản phẩm, thêm vào giỏ hàng, và thực hiện thanh toán. Mongoose quản lý dữ liệu sản phẩm, giỏ hàng, và đơn hàng. Express.js xây dựng API và xử lý các yêu cầu HTTP.

## 4. Hệ Thống Quản Lý Người Dùng (User Management Systems)

**Ví dụ:** Hệ thống quản lý người dùng

- **Công nghệ:** Node.js, Express.js, MongoDB, Mongoose, bcrypt
- **Mô tả:** Hệ thống cho phép người dùng đăng ký, đăng nhập, và quản lý hồ sơ cá nhân. Mongoose quản lý dữ liệu người dùng và quyền truy cập. bcrypt được sử dụng để mã hóa mật khẩu.

## 5. Ứng Dụng API RESTful (RESTful API Applications)

**Ví dụ:** API RESTful cho quản lý sản phẩm

- **Công nghệ:** Node.js, Express.js, MongoDB, Mongoose
- **Mô tả:** API cho phép thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên dữ liệu sản phẩm. Mongoose quản lý dữ liệu sản phẩm và Express.js xử lý các yêu cầu API.

## 6. Ứng Dụng Quản Lý Dự Án (Project Management Applications)

**Ví dụ:** Ứng dụng quản lý dự án

- **Công nghệ:** Node.js, Express.js, MongoDB, Mongoose
- **Mô tả:** Ứng dụng cho phép người dùng tạo và quản lý các dự án, nhiệm vụ, và thành viên nhóm. Mongoose quản lý dữ liệu dự án và nhiệm vụ. Express.js xây dựng API và xử lý các yêu cầu HTTP.

## 7. Ứng Dụng Đặt Phòng Khách Sạn (Hotel Booking Applications)

**Ví dụ:** Ứng dụng đặt phòng khách sạn

- **Công nghệ:** Node.js, Express.js, MongoDB, Mongoose
- **Mô tả:** Ứng dụng cho phép người dùng tìm kiếm và đặt phòng khách sạn. Mongoose quản lý dữ liệu khách sạn và đặt phòng. Express.js xử lý các yêu cầu HTTP và xây dựng API.

## 8. Ứng Dụng Quản Lý Sự Kiện (Event Management Applications)

**Ví dụ:** Ứng dụng quản lý sự kiện

- **Công nghệ:** Node.js, Express.js, MongoDB, Mongoose

- **Mô tả:** Ứng dụng cho phép người dùng tạo và quản lý các sự kiện, đăng ký tham gia, và theo dõi thông tin sự kiện. Mongoose quản lý dữ liệu sự kiện và người tham gia. Express.js xây dựng API và xử lý các yêu cầu HTTP.

# NỘI DUNG CHI TIẾT

## PHẦN 1: NoSQL Best Practices (10.1)

### 1. NoSQL là gì?

- Không giống SQL (quan hệ, dùng bảng), NoSQL lưu trữ dữ liệu dưới dạng **document**, **key-value**, **graph**, hoặc **column-based**.
- Phù hợp cho **dữ liệu linh hoạt, thay đổi thường xuyên**, và các hệ thống cần mở rộng quy mô lớn (như Facebook, Amazon).

### 2. Best Practices:

- **Thiết kế dữ liệu dựa trên cách dùng thực tế (query-driven design)**  
→ Nghĩ đến các câu truy vấn trước khi thiết kế schema.
- **Tránh join phức tạp**, nên gộp dữ liệu vào cùng document nếu có thể.
- **Dùng index hợp lý** để tối ưu truy vấn.
- **Chọn đúng kiểu NoSQL**: MongoDB (document), Redis (key-value), Neo4j (graph).

### Bài tập:

- Vẽ sơ đồ thiết kế dữ liệu đơn giản cho hệ thống blog gồm: Users, Posts, Comments.
- Hãy thử gộp comment vào post, hoặc lưu riêng và liên kết qua postId.

### Mongo Shell là gì?

- Là công cụ dòng lệnh để tương tác trực tiếp với MongoDB.
- Giúp bạn thêm, sửa, xóa, truy vấn dữ liệu nhanh chóng.

### Một số lệnh cơ bản:

```
bash
```

```
# Kết nối đến database
mongo
```

```
# Xem database
show dbs
```

```
# Chọn database
use blog
```

```
# Tạo/bổ sung document
db.users.insertOne({ name: "Alice", age: 25 })
```

```
# Truy vấn
db.users.find({ age: { $gt: 20 } })
```

```
# Cập nhật
db.users.updateOne({ name: "Alice" }, { $set: { age: 26 } })
```



```
# Xóa
db.users.deleteOne({ name: "Alice" })
```

### ✅ Bài tập:

- Tạo một database `blogDB`.
- Thêm 2 user vào collection `users`.
- Truy vấn tất cả user có tuổi trên 20.

## 🔗 PHẦN 3: Node.js và MongoDB (10.3)

### 💡 Mục tiêu:

Kết nối ứng dụng Node.js đến MongoDB và thực hiện các thao tác CRUD (Create, Read, Update, Delete).

### ✂ Cài đặt:

```
bash
```

```
npm init -y
npm install mongodb
```

### 📘 Mã mẫu kết nối:

```
js

const { MongoClient } = require('mongodb');
const uri = "mongodb://localhost:27017";
const client = new MongoClient(uri);

async function run() {
  try {
    await client.connect();
    const db = client.db("blogDB");
    const users = db.collection("users");
    await users.insertOne({ name: "Bob", age: 22 });
    console.log("User inserted!");
  } finally {
    await client.close();
  }
}

run();
```

### ✅ Bài tập:

- Cài Node.js + MongoDB.
- Tạo file `app.js` và kết nối đến MongoDB local.
- Thêm một user mới.

## PHẦN 4: Mongoose và Mongoose Population (11 & 12)

### MongoDB và Tài Liệu (Documents)

- **MongoDB** lưu trữ dữ liệu dưới dạng tài liệu (documents) trong các collection.
- **Tài liệu** trong MongoDB không có cấu trúc cố định, nghĩa là bất kỳ tài liệu nào cũng có thể được lưu trữ trong bất kỳ collection nào mà không cần tuân theo một cấu trúc nhất định.

### Mongoose ODM

- **Mongoose** là một thư viện ODM (Object Data Modeling) cho MongoDB và Node.js.
- **ODM** là một kỹ thuật ánh xạ các đối tượng trong mã nguồn (object) với các tài liệu trong cơ sở dữ liệu (document).

### Lợi Ích của Mongoose ODM

- **Thêm cấu trúc cho tài liệu MongoDB:** Mongoose thêm cấu trúc cho các tài liệu MongoDB thông qua schema.
- **Kiểm tra và xác thực dữ liệu:** Mongoose giúp kiểm tra và xác thực dữ liệu trước khi lưu vào cơ sở dữ liệu.
- **Mô hình hóa dữ liệu:** Mongoose giúp mô hình hóa dữ liệu, giúp việc quản lý và thao tác dữ liệu trở nên dễ dàng hơn.

### Cách Mongoose Hoạt Động

- **Schema:** Định nghĩa cấu trúc của các tài liệu trong một collection MongoDB. Mỗi schema mô tả các trường (fields) và kiểu dữ liệu (data types) của chúng.
- **Model:** Áp dụng schema cho các tài liệu và chịu trách nhiệm cho các thao tác CRUD (Create, Read, Update, Delete).

### Ví Dụ về Schema và Model

- **Schema:**

```
const blogSchema = new Schema({
  title: String,
  slug: String,
  published: Boolean,
  author: String,
  content: String,
  tags: [String],
  createdAt: Date,
  updatedAt: Date,
  comments: [{ user: String, content: String, votes: Number }]
});
```

- **Model:**

```
const Blog = mongoose.model('Blog', blogSchema);
```

### Tại Sao Sử Dụng Mongoose?

- **Mô hình dữ liệu linh hoạt của MongoDB:** MongoDB cho phép thay đổi và cập nhật dữ liệu dễ dàng.
- **Schema bán cứng của Mongoose:** Mongoose ép buộc một schema bán cứng ngay từ đầu, giúp duy trì cấu trúc dữ liệu và đảm bảo tính nhất quán.

## Kết Nối với MongoDB

- **Cài đặt Mongoose:** `npm i mongoose`
- **Kết nối với MongoDB:**

```

• const mongoose = require('mongoose');
• const connectDB = async () => {
•   try {
•     const connect = await mongoose.connect(process.env.MONGO_URI);
•     console.log(`MongoDB Connected: ${connect.connection.host}`);
•   } catch (err) {
•     console.log(err);
•     process.exit(1);
•   }
• }
• module.exports = connectDB;

```

## Tóm Lại

- **Mongoose ODM** giúp thêm cấu trúc cho các tài liệu MongoDB, kiểm tra và xác thực dữ liệu, và mô hình hóa dữ liệu.
- **Schema** định nghĩa cấu trúc của các tài liệu, trong khi **Model** áp dụng schema và chịu trách nhiệm cho các thao tác CRUD

## Thiết lập môi trường cho Mongoose

### Tóm tắt

- **Cài đặt Node.js và npm:** Tải và cài đặt từ trang web chính thức.
- **Tạo thư mục dự án:** Tạo và chuyển vào thư mục dự án.
- **Khởi tạo dự án Node.js:** Sử dụng `npm init -y`.
- **Cài đặt Mongoose:** Sử dụng `npm install mongoose`.
- **Cài đặt Nodemon:** Sử dụng `npm install -D nodemon`.
- **Cài đặt dotenv:** Sử dụng `npm install dotenv`.
- **Thiết lập script trong package.json:** Thêm script để chạy dự án với Nodemon.
- **Tạo file .env:** Lưu các biến môi trường.
- **Tạo file server.js:** Khởi động server và kết nối với MongoDB.
- **Chạy dự án:** Sử dụng `npm run dev`.

## Bước 1: Cài Đặt Node.js và npm

Trước tiên, bạn cần cài đặt Node.js và npm (Node Package Manager) nếu chưa có. Bạn có thể tải và cài đặt từ trang web chính thức của Node.js: Node.js.

## Bước 2: Tạo Thư Mục Dự Án

Tạo một thư mục mới cho dự án của bạn và chuyển vào thư mục đó:

```
mkdir my-mongoose-project  
cd my-mongoose-project
```

## Bước 3: Khởi Tạo Dự Án Node.js

Khởi tạo một dự án Node.js mới bằng cách sử dụng npm:

```
npm init -y
```

Lệnh này sẽ tạo ra một file `package.json` với các thiết lập mặc định.

## Bước 4: Cài Đặt Mongoose

Cài đặt Mongoose bằng npm:

```
npm install mongoose
```

## Bước 5: Cài Đặt Nodemon

Nodemon là một công cụ giúp tự động khởi động lại server khi có thay đổi trong mã nguồn. Cài đặt Nodemon như một phụ thuộc phát triển:

```
npm install -D nodemon
```

## Bước 6: Cài Đặt dotenv

dotenv là một thư viện giúp quản lý các biến môi trường trong dự án. Cài đặt dotenv:

```
npm install dotenv
```

## Bước 7: Thiết Lập Script trong package.json

Thêm một script để chạy dự án với Nodemon trong file `package.json`:

```
"scripts": {
```

```
"dev": "nodemon server.js"
}
```

## Bước 8: Tạo File .env

Tạo một file .env trong thư mục gốc của dự án để lưu các biến môi trường:

```
NODE_ENV=development
PORT=8000
MONGO_URI=mongodb://127.0.0.1:27017/mydatabase
```

## Bước 9: Tạo File server.js

Tạo một file server.js để khởi động server và kết nối với MongoDB:

```
const express = require('express');
const mongoose = require('mongoose');
require('dotenv').config();

const app = express();
const port = process.env.PORT || 5000;

// Kết nối với MongoDB
mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => {
  console.log('MongoDB connected');
}).catch(err => {
  console.error('MongoDB connection error:', err);
});

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Route mẫu
app.get('/', (req, res) => {
  res.send('Hello, Mongoose!');
});

// Khởi động server
app.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});
```

## Bước 10: Chạy Dự Án

Chạy dự án bằng lệnh sau:

```
npm run dev
```

Nodemon sẽ khởi động server và tự động khởi động lại khi có thay đổi trong mã nguồn.

## Cách Mongoose Hoạt Động

- **Schema** định nghĩa cấu trúc của các tài liệu trong MongoDB.
- **Model** áp dụng schema và chịu trách nhiệm cho các thao tác CRUD.
- **Kết nối với MongoDB** để sử dụng Mongoose.
- **Thực hiện các thao tác CRUD** để quản lý dữ liệu trong MongoDB.

### 1. Định Nghĩa Schema

- **Schema** trong Mongoose định nghĩa cấu trúc của các tài liệu trong một collection MongoDB. Mỗi schema mô tả các trường (fields) và kiểu dữ liệu (data types) của chúng.
- Ví dụ về định nghĩa schema cho một blog:

```
const mongoose = require('mongoose');
const { Schema } = mongoose;
•
const blogSchema = new Schema({
  title: String,
  slug: String,
  published: Boolean,
  author: String,
  content: String,
  tags: [String],
  createdAt: Date,
  updatedAt: Date,
  comments: [{ user: String, content: String, votes: Number }]
});
```

### 2. Tạo Model từ Schema

- **Model** trong Mongoose là một lớp (class) được xây dựng dựa trên schema và đại diện cho collection trong MongoDB.
- Model chịu trách nhiệm cho các thao tác CRUD (Create, Read, Update, Delete) trên các tài liệu.
- Ví dụ về tạo model từ schema:

```
const Blog = mongoose.model('Blog', blogSchema);
```

### 3. Kết Nối với MongoDB

- Để sử dụng Mongoose, bạn cần kết nối với cơ sở dữ liệu MongoDB.
- Ví dụ về kết nối với MongoDB:

```
mongoose.connect('mongodb://127.0.0.1:27017/mydatabase', {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => {
  console.log('MongoDB connected');
}).catch(err => {
```

```
• console.error('MongoDB connection error:', err);
• });
```

#### 4. Thực Hiện Các Thao Tác CRUD

- **Create:** Tạo và lưu một tài liệu mới vào MongoDB.

```
• const newBlog = new Blog({
•   title: 'My First Blog',
•   slug: 'my-first-blog',
•   published: true,
•   author: 'John Doe',
•   content: 'This is the content of my first blog.',
•   tags: ['introduction', 'first'],
•   createdAt: new Date(),
•   updatedAt: new Date(),
•   comments: []
• });
•
• newBlog.save()
•   .then(() => console.log('Blog saved successfully'))
•   .catch(err => console.error('Error saving blog:', err));
```

- **Read:** Truy xuất tài liệu từ MongoDB.

```
• Blog.findOne({ slug: 'my-first-blog' })
•   .then(blog => console.log('Blog found:', blog))
•   .catch(err => console.error('Error finding blog:', err));
```

- **Update:** Cập nhật tài liệu trong MongoDB.

```
• Blog.findOneAndUpdate({ slug: 'my-first-blog' }, { title: 'Updated
  Blog Title' }, { new: true })
•   .then(updatedBlog => console.log('Blog updated:', updatedBlog))
•   .catch(err => console.error('Error updating blog:', err));
```

- **Delete:** Xóa tài liệu khỏi MongoDB.

```
• Blog.findOneAndDelete({ slug: 'my-first-blog' })
•   .then(() => console.log('Blog deleted successfully'))
•   .catch(err => console.error('Error deleting blog:', err));
```

## Minh Họa Cụ Thể

### Bước 1: Định Nghĩa Schema

```
const mongoose = require('mongoose');
const { Schema } = mongoose;

const blogSchema = new Schema({
  title: String,
  slug: String,
```

```
published: Boolean,
author: String,
content: String,
tags: [String],
createdAt: Date,
updatedAt: Date,
comments: [{ user: String, content: String, votes: Number }]
});
```

## Bước 2: Tạo Model từ Schema

```
const Blog = mongoose.model('Blog', blogSchema);
```

## Bước 3: Kết Nối với MongoDB

```
mongoose.connect('mongodb://127.0.0.1:27017/mydatabase', {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => {
  console.log('MongoDB connected');
}).catch(err => {
  console.error('MongoDB connection error:', err);
});
```

## Bước 4: Thực Hiện Các Thao Tác CRUD

- **Create:**

```
• const newBlog = new Blog({
•   title: 'My First Blog',
•   slug: 'my-first-blog',
•   published: true,
•   author: 'John Doe',
•   content: 'This is the content of my first blog.',
•   tags: ['introduction', 'first'],
•   createdAt: new Date(),
•   updatedAt: new Date(),
•   comments: []
• });
•
• newBlog.save()
•   .then(() => console.log('Blog saved successfully'))
•   .catch(err => console.error('Error saving blog:', err));
```

- **Read:**

```
• Blog.findOne({ slug: 'my-first-blog' })
•   .then(blog => console.log('Blog found:', blog))
•   .catch(err => console.error('Error finding blog:', err));
```

- **Update:**

```
• Blog.findOneAndUpdate({ slug: 'my-first-blog' }, { title: 'Updated
  Blog Title' }, { new: true })
•   .then(updatedBlog => console.log('Blog updated:', updatedBlog))
```



- `.catch(err => console.error('Error updating blog:', err));`
- **Delete:**

- `Blog.findOneAndDelete({ slug: 'my-first-blog' })`
- `.then(() => console.log('Blog deleted successfully'))`
- `.catch(err => console.error('Error deleting blog:', err));`

## MVC (Model-View-Controller) trong Mongoose

### 1. Model (Mô Hình)

- **Model** đại diện cho cấu trúc dữ liệu trong cơ sở dữ liệu MongoDB.
- Mỗi model được xây dựng trên một schema Mongoose, mô tả cấu trúc của các tài liệu trong một collection.
- Model chịu trách nhiệm cho các thao tác CRUD (Create, Read, Update, Delete) trên các tài liệu.

Ví dụ về định nghĩa schema và tạo model:

```
const mongoose = require('mongoose');
const { Schema } = mongoose;

// Định nghĩa schema cho blog
const blogSchema = new Schema({
  title: String,
  content: String,
  author: String,
  createdAt: Date
});

// Tạo model từ schema
const Blog = mongoose.model('Blog', blogSchema);
```

### 2. View (Giao Diện)

- **View** là thành phần hiển thị dữ liệu cho người dùng.
- Trong một ứng dụng Express.js, Views thường là các file template (e.g. EJS, Handlebars, Pug) được render bởi server và gửi dưới dạng HTML tới client.

Ví dụ về một file template EJS để hiển thị danh sách các bài viết blog:

```
<!-- views/blogPosts.ejs -->
<!DOCTYPE html>
<html>
```

```
<head><title>Blog Posts</title></head>
<body>
  <h1>Blog Posts</h1>
  <ul>
    <% posts.forEach(post => { %>
      <li><%= post.title %> by <%= post.author %></li>
    <% }) %>
  </ul>
</body>
</html>
```

### 3. Controller (Bộ Điều Khiển)

- **Controller** sử dụng Model để truy xuất dữ liệu và quyết định View nào sẽ hiển thị dữ liệu đó.
- Controller xử lý logic ứng dụng, nhận các yêu cầu từ người dùng và trả về các phản hồi.

Ví dụ về một controller để lấy danh sách các bài viết blog và render view:

```
const Blog = require('../models/blog');

// Controller để lấy danh sách các bài viết blog
exports.getBlogPosts = async (req, res) => {
  try {
    const posts = await Blog.find();
    res.render('blogPosts', { posts }); // Sử dụng View để hiển thị các bài
viết
  } catch (error) {
    console.error('Error fetching blog posts:', error);
    res.status(500).send('Error fetching blog posts');
  }
};
```

## Minh Họa Cụ Thể

### Bước 1: Định Nghĩa Model

Tạo file `models/blog.js` và định nghĩa schema cùng model cho blog:

```
const mongoose = require('mongoose');
const { Schema } = mongoose;

const blogSchema = new Schema({
  title: String,
  content: String,
  author: String,
  createdAt: Date
});
```

```
const Blog = mongoose.model('Blog', blogSchema);
module.exports = Blog;
```

## Bước 2: Tạo Controller

Tạo file `controllers/blogController.js` và định nghĩa controller để lấy danh sách các bài viết blog:

```
const Blog = require('../models/blog');

exports.getBlogPosts = async (req, res) => {
  try {
    const posts = await Blog.find();
    res.render('blogPosts', { posts });
  } catch (error) {
    console.error('Error fetching blog posts:', error);
    res.status(500).send('Error fetching blog posts');
  }
};
```

## Bước 3: Tạo View

Tạo thư mục `views` và file `blogPosts.ejs` để hiển thị danh sách các bài viết blog:

```
<!-- views/blogPosts.ejs -->
<!DOCTYPE html>
<html>
<head><title>Blog Posts</title></head>
<body>
  <h1>Blog Posts</h1>
  <ul>
    <% posts.forEach(post => { %>
      <li><%= post.title %> by <%= post.author %></li>
    <% }) %>
  </ul>
</body>
</html>
```

## Bước 4: Thiết Lập Route

Tạo file `routes/blogRoutes.js` và thiết lập route để sử dụng controller:

```
const express = require('express');
const router = express.Router();
const blogController = require('../controllers/blogController');

router.get('/blogs', blogController.getBlogPosts);
```

```
module.exports = router;
```

## Bước 5: Khởi Động Server

Tạo file `server.js` để khởi động server và kết nối với MongoDB:

```
const express = require('express');
const mongoose = require('mongoose');
const blogRoutes = require('./routes/blogRoutes');
require('dotenv').config();

const app = express();
const port = process.env.PORT || 5000;

mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => {
  console.log('MongoDB connected');
}).catch(err => {
  console.error('MongoDB connection error:', err);
});

app.set('view engine', 'ejs');
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(blogRoutes);

app.listen(port, () => {
  console.log(`Server listening on port ${port}`);
});
```

## Tóm Lại

- **Model:** Đại diện cho cấu trúc dữ liệu và chịu trách nhiệm cho các thao tác CRUD.
- **View:** Hiển thị dữ liệu cho người dùng, thường là các file template.
- **Controller:** Xử lý logic ứng dụng, sử dụng Model để truy xuất dữ liệu và quyết định View nào sẽ hiển thị dữ liệu đó.

## KINH NGHIỆM THỰC TIỄN, VẤN ĐỀ CẦN LƯU Ý, VÀ CÁC MẸO (TIPS)

### 1. KINH NGHIỆM THỰC TIỄN

#### Từ dự án thực tế:

- **Thiết kế dữ liệu MongoDB cần có tầm nhìn dài hạn:** Dữ liệu trong MongoDB rất linh hoạt, nhưng nếu không tính trước cách truy vấn sẽ dễ dẫn đến lỗi hoặc hiệu suất kém.
- **Dùng Mongoose giúp tiết kiệm thời gian debug:** Vì schema định nghĩa rõ ràng, bạn dễ phát hiện lỗi sai kiểu dữ liệu hoặc thiếu trường.
- **Populate cực kỳ hữu ích khi hiển thị dữ liệu liên quan:** VD: Blog, diễn đàn, eCommerce → luôn phải hiển thị user, sản phẩm, bình luận liên kết.

#### Trong teamwork:

- Mỗi model nên để riêng file (`user.model.js`, `comment.model.js`) và tổ chức thành thư mục `models`.
- Luôn viết một vài **test case CRUD đơn giản** sau khi tạo model/schema để chắc chắn hoạt động đúng.

### 2. VẤN ĐỀ CẦN LƯU Ý

| Vấn đề                                       | Lưu ý                                                                                                                  |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Thiết kế dữ liệu sai                         | Nếu không xác định rõ quan hệ dữ liệu từ đầu, về sau sẽ rất khó chỉnh sửa, đặc biệt với NoSQL.                         |
| Lạm dụng populate                            | Quá nhiều <code>.populate()</code> có thể khiến truy vấn chậm, nên chỉ dùng khi thực sự cần hiển thị dữ liệu liên kết. |
| Không kiểm tra lỗi khi kết nối DB            | Dễ gây crash ứng dụng nếu MongoDB không kết nối được → luôn dùng <code>try/catch</code> .                              |
| Thiếu validation dữ liệu đầu vào             | Dễ gây lỗi logic hoặc dữ liệu không đúng (user không có tên/email) → dùng Mongoose <code>schema validation</code> .    |
| Không index trường hay tìm kiếm thường xuyên | Truy vấn chậm khi dữ liệu lớn – cần thêm <code>.createIndex()</code> hoặc dùng <code>index: true</code> trong schema.  |

### 3. MẸO HAY (TIPS)

#### MongoDB & Mongo Shell:

- Dùng `db.collection.find().pretty()` để hiển thị kết quả dễ đọc.
- Dùng `db.collection.findOne()` thay vì `.find()[0]` để lấy 1 bản ghi.
- Có thể dùng `ObjectId()` để tìm kiếm bằng `_id`.

#### Node.js + MongoDB:

- Luôn dùng `async/await` kết hợp `try/catch` thay vì `callback` → mã dễ đọc hơn, ít lỗi.
- Cấu hình `dotenv` để lưu URI kết nối MongoDB tách khỏi mã nguồn.

#### Mongoose:

- Dùng `.lean()` sau `.find()` nếu bạn chỉ muốn lấy dữ liệu thuần (giúp tăng hiệu suất).

js

```
const users = await User.find().lean(); // nhanh hơn .find() thông thường
```

- Khi schema phức tạp, chia nhỏ schema (subdocuments) hoặc tạo model riêng.

## Populate:

- Bạn có thể **lồng nhiều cấp populate**, nhưng nên hạn chế số cấp nếu không cần thiết:

js

```
Post.find().populate({
  path: 'comments',
  populate: { path: 'user' }
});
```

- Dùng `.select()` để chỉ lấy những trường cần thiết khi populate:

js

```
.populate('user', 'name email') // chỉ lấy name và email
```

## TÓM TẮT NHANH

| Phần               | Kinh nghiệm                | Vấn đề                                 | Mẹo                                                                                          |
|--------------------|----------------------------|----------------------------------------|----------------------------------------------------------------------------------------------|
| <b>NoSQL</b>       | Thiết kế theo truy vấn     | Lưu trữ dư thừa                        | Chia nhỏ document hợp lý                                                                     |
| <b>Mongo Shell</b> | Kiểm tra trực tiếp dữ liệu | Dễ sai lệnh khi không rõ cấu trúc      | Dùng <code>.pretty()</code> , <code>ObjectId()</code>                                        |
| <b>Node.js</b>     | CRUD hiệu quả              | Crash nếu không <code>try/catch</code> | Dùng <code>async/await</code> , tách URI ra file <code>.env</code>                           |
| <b>Mongoose</b>    | Kiểm soát dữ liệu tốt      | Lỗi nếu schema sai                     | <code>.lean()</code> , <code>required</code> , <code>default</code> , <code>.select()</code> |
| <b>Populate</b>    | Hiển thị dữ liệu liên quan | Chậm nếu quá lạm dụng                  | Dùng <code>.populate('user', 'name')</code>                                                  |

## PHÂN TÍCH QUY LUẬT THỰC THI

PROMT: Tôi là người mới học, tôi cần phát hiện quy luật thực thi của chương trình. Hãy Mô tả quá trình thực thi của code của bài tập

### EXERCISE

#### DỰ ÁN: Ứng dụng Blog Đơn Giản (Node.js + MongoDB)

#### I. TÓM TẮT MỤC TIÊU

- Vận dụng kiến thức Mongoose để xây dựng các schema chuẩn và mối quan hệ giữa collection.
- Thực hành CRUD (Create - Read - Update - Delete).
- Sử dụng `.populate()` để liên kết và truy xuất dữ liệu liên quan.
- Xây dựng ứng dụng backend có cấu trúc chuyên nghiệp, sẵn sàng mở rộng.

#### II. CÁC CHỨC NĂNG CỦA ỨNG DỤNG

| Chức năng                | Mô tả                                                               |
|--------------------------|---------------------------------------------------------------------|
| Đăng ký người dùng       | Tạo người dùng mới                                                  |
| Tạo bài viết             | Người dùng đăng bài viết                                            |
| Bình luận bài viết       | Người dùng bình luận vào bài viết                                   |
| Lấy danh sách bài viết   | Bao gồm thông tin người viết và các bình luận (kèm người bình luận) |
| Xóa bài viết / bình luận | Thao tác CRUD đầy đủ                                                |

#### III. CÔNG NGHỆ SỬ DỤNG

- **Node.js** – môi trường chạy
- **Express.js** – web framework
- **MongoDB** – cơ sở dữ liệu NoSQL
- **Mongoose** – ODM dùng định nghĩa schema
- **dotenv** – quản lý biến môi trường
- **Postman** – kiểm thử API

#### IV. CẤU TRÚC THƯ MỤC

pgsql

```
blog-app/  
├── models/  
│   ├── user.model.js  
│   ├── post.model.js  
│   └── comment.model.js  
├── routes/  
│   ├── user.routes.js  
│   ├── post.routes.js  
│   └── comment.routes.js  
└── controllers/ (tùy chọn)
```

```
|— config/
|   |— db.js
|— .env
|— app.js
|— package.json
```

## V. TÓM TẮT CÁC BƯỚC THỰC HIỆN

| Bước | Mô tả                                           |
|------|-------------------------------------------------|
| 1    | Khởi tạo dự án Node.js                          |
| 2    | Cài đặt các thư viện cần thiết                  |
| 3    | Kết nối MongoDB bằng Mongoose                   |
| 4    | Tạo model/schema: User, Post, Comment           |
| 5    | Tạo các route và controller cho CRUD            |
| 6    | Sử dụng <code>.populate()</code> trong truy vấn |
| 7    | Test API với Postman                            |
| 8    | Hoàn thiện và kiểm thử                          |

## VI. CHI TIẾT TỪNG BƯỚC

### ◆ BƯỚC 1: Khởi tạo dự án

bash

```
mkdir blog-app
cd blog-app
npm init -y
```

### ◆ BƯỚC 2: Cài đặt thư viện

bash

```
npm install express mongoose dotenv
npm install --save-dev nodemon
```

**Thêm vào `package.json` để chạy dễ hơn:**

json

```
"scripts": {
  "start": "node app.js",
  "dev": "nodemon app.js"
}
```



## ◆ BƯỚC 3: Kết nối MongoDB

### Tạo file .env:

```
ini

MONGODB_URI=mongodb://localhost:27017/blogDB
PORT=3000
```

### Tạo file config/db.js:

```
js

const mongoose = require('mongoose');
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log('MongoDB connected');
  } catch (err) {
    console.error(err);
    process.exit(1);
  }
};
module.exports = connectDB;
```

## ◆ BƯỚC 4: Tạo các Mongoose Models

### models/user.model.js:

```
js

const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
  name: String,
  email: String
});
module.exports = mongoose.model('User', userSchema);
```

### models/post.model.js:

```
js

const mongoose = require('mongoose');
const postSchema = new mongoose.Schema({
  title: String,
  content: String,
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
});
module.exports = mongoose.model('Post', postSchema);
```

### models/comment.model.js:

```
js

const mongoose = require('mongoose');
const commentSchema = new mongoose.Schema({
  text: String,
  post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post' },
```

```

    user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
  });
module.exports = mongoose.model('Comment', commentSchema);

```

## ◆ BƯỚC 5: Tạo Routes & CRUD API

Ví dụ với `routes/post.routes.js`:

```

js

const express = require('express');
const router = express.Router();
const Post = require('../models/post.model');
const Comment = require('../models/comment.model');

// Tạo bài viết
router.post('/', async (req, res) => {
  const { title, content, author } = req.body;
  const post = new Post({ title, content, author });
  await post.save();
  res.json(post);
});

// Lấy tất cả bài viết (kèm comment và user)
router.get('/', async (req, res) => {
  const posts = await Post.find()
    .populate('author', 'name')
    .populate({
      path: 'comments',
      populate: { path: 'user', select: 'name' }
    });
  res.json(posts);
});

module.exports = router;

```

Tương tự, bạn tạo các route cho `user.routes.js` và `comment.routes.js`.

## ◆ BƯỚC 6: Tích hợp vào app chính

`app.js`:

```

js

require('dotenv').config();
const express = require('express');
const connectDB = require('./config/db');

const app = express();
app.use(express.json());

// Kết nối DB
connectDB();

// Routes
app.use('/users', require('./routes/user.routes'));
app.use('/posts', require('./routes/post.routes'));
app.use('/comments', require('./routes/comment.routes'));

const PORT = process.env.PORT || 3000;

```

```
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

## BUỚC 7: Test API với Postman

Thực hiện các bước:

1. POST /users → tạo người dùng
2. POST /posts → tạo bài viết (đưa userId vào)
3. POST /comments → bình luận vào bài viết
4. GET /posts → xem bài viết kèm bình luận và người bình luận (.populate())

## ◆ BUỚC 8: Hoàn thiện, kiểm tra, sửa lỗi

- Kiểm tra dữ liệu trong MongoDB bằng Mongo Shell hoặc Compass.
- Xử lý lỗi với try/catch.
- Đảm bảo validate dữ liệu (sử dụng Mongoose schema options: required, default, v.v.)

## ✓ VII. KẾT QUẢ CUỐI CÙNG CẦN ĐẠT ĐƯỢC

- Hiểu rõ quy trình xây dựng backend Node.js + MongoDB.
- Áp dụng .populate() đúng cách.
- Biết tổ chức mã nguồn có cấu trúc rõ ràng.
- Có thể mở rộng thêm chức năng như: xác thực, phân trang, tìm kiếm.

## LUYỆN TẬP

### BÀI TẬP THỰC HÀNH 01: Ứng dụng Quản lý Bài viết và Bình luận

#### ✓ 1. Tên bài tập:

**Xây dựng hệ thống Blog đơn giản với User, Post và Comment**

#### 2. Mục đích của bài tập:

- Luyện tập sử dụng MongoDB với Node.js.
- Hiểu rõ cách dùng Mongoose và xây dựng mối quan hệ giữa các collection.
- Thực hành .populate() để lấy dữ liệu liên quan.
- Áp dụng kiến thức CRUD, schema validation và tổ chức dự án backend cơ bản.

### 3. Tóm tắt nội dung và bài tập:

Bạn sẽ xây dựng một hệ thống **Blog đơn giản** bao gồm:

- Người dùng (**User**)
- Bài viết (**Post**)
- Bình luận (**Comment**)

Người dùng có thể tạo bài viết và bình luận vào bài viết. Mỗi bình luận sẽ liên kết với người viết và bài viết.

### 4. Yêu cầu của bài tập:

- Tạo 3 model: `User`, `Post`, `Comment` với các schema tương ứng.
- Tạo API để CRUD người dùng, bài viết và bình luận.
- Dùng **Mongoose populate** để:
  - Khi lấy bài viết → hiển thị bình luận đi kèm và thông tin người viết bình luận.
  - Khi lấy bình luận → hiển thị thông tin người viết.

### 5. Các chức năng cần có:

| Chức năng                   | Mô tả                                                   |
|-----------------------------|---------------------------------------------------------|
| Đăng ký người dùng          | Tạo user mới                                            |
| Tạo bài viết                | Người dùng đăng bài viết                                |
| Đăng bình luận              | Người dùng bình luận vào bài viết                       |
| Xem tất cả bài viết         | Hiển thị bài viết, tác giả, bình luận và user bình luận |
| Xóa bài viết hoặc bình luận | Cho phép xóa bài viết hoặc comment theo id              |

### 6. Công nghệ sử dụng:

- **Node.js** (Runtime)
- **Express.js** (Framework)
- **MongoDB** (Cơ sở dữ liệu NoSQL)
- **Mongoose** (ODM)
- **Postman** hoặc **Thunder Client** để test API
- (Tùy chọn: `dotenv`, `nodemon`)

### 7. Cấu trúc dự án gợi ý:

`pgsql`

`blog-app/`

```
├── models/
│   ├── user.model.js
│   ├── post.model.js
│   └── comment.model.js
```

```

|— routes/
|   |— user.routes.js
|   |— post.routes.js
|   |— comment.routes.js
|— controllers/
|   |— (tùy chọn)
|— app.js
|— .env
|— package.json

```

## 8. Các bước cần thực hiện:

1. **Khởi tạo project Node.js** bằng `npm init -y`.
2. **Cài đặt thư viện cần thiết:**  
`npm install express mongoose dotenv`
3. **Kết nối MongoDB** với Mongoose trong `app.js`.
4. **Tạo schema và model** cho User, Post, Comment.
5. **Thiết lập API routes** cho CRUD với Express.
6. **Áp dụng `.populate()`** trong các truy vấn liên quan.
7. **Test API bằng Postman** để chắc chắn logic hoạt động.
8. **Viết 3 test case** (xem mục cuối).
9. (Tùy chọn): Tách controller, viết middleware validate.

## 9. Kết quả cần đạt được:

- Người học hiểu được cách **xây dựng và tổ chức backend đơn giản** với Node.js và MongoDB.
- Thực hành thành thạo Mongoose: schema, model, validation, populate.
- Gửi và nhận dữ liệu dạng JSON thông qua REST API.
- Vận dụng tốt mối quan hệ giữa các collection trong MongoDB.

## 10. Tạo 3 Test Case mẫu:

### Test Case 1: Tạo một người dùng mới

- **Input:**

```

json

{
  "name": "Nguyen Van A",
  "email": "a@example.com"
}

```

- **Expected Output:**

- Trả về thông tin user mới với `_id`, `name`, `email`.

## ✅ Test Case 2: Tạo bài viết và comment liên kết

- **Input:**
  - Tạo bài viết với `userId` vừa tạo.
  - Tạo comment gắn với bài viết đó và `userId`.
- **Expected Output:**
  - Bài viết chứa mảng comment với nội dung và thông tin người bình luận (dùng `populate`).

## ✅ Test Case 3: Lấy tất cả bài viết (kèm bình luận đã populate)

- **API:** `GET /posts`
- **Expected Output:**
  - Trả về danh sách bài viết có:
    - Tên người viết
    - Comment (nội dung + người bình luận)

## BÀI TẬP 02: Ứng dụng Quản lý Khóa học và Người học

### ✅ 1. Tên bài tập:

Quản lý Học viên và Khóa học

### 2. Mục đích:

- Luyện tập mô hình **quan hệ nhiều-nhiều** (many-to-many) giữa `User` và `Course`.
- Sử dụng `.populate()` để truy xuất các khóa học mà một học viên đang học và danh sách học viên của một khóa học.

### 3. Tóm tắt nội dung:

Tạo một ứng dụng quản lý:

- Người học (**User**)
- Khóa học (**Course**)

Một học viên có thể đăng ký nhiều khóa học. Một khóa học có thể có nhiều học viên.

### 4. Yêu cầu:

- Tạo model `User` và `Course`.
- Mỗi `User` có trường `enrolledCourses` (mảng `ObjectId` tham chiếu đến `Course`).
- Mỗi `Course` có trường `students` (mảng `ObjectId` tham chiếu đến `User`).
- Tạo API để:
  - Tạo học viên, khóa học.
  - Học viên đăng ký khóa học.

- Lấy danh sách học viên theo khóa học (dùng `.populate()`).
- Lấy danh sách khóa học của học viên (dùng `.populate()`).

## 5. Chức năng:

- Tạo người học
- Tạo khóa học
- Đăng ký học
- Xem danh sách khóa học/học viên (dùng `populate`)

## 6. Công nghệ:

- Node.js
- Express
- MongoDB
- Mongoose

## 7. Cấu trúc gợi ý:

pgsql

```
course-app/
├── models/
│   ├── user.model.js
│   └── course.model.js
├── routes/
│   ├── user.routes.js
│   └── course.routes.js
├── app.js
└── .env
```

## 8. Các bước thực hiện:

1. Cài đặt dự án như bài 1.
2. Tạo schema `User` (có trường `enrolledCourses`).
3. Tạo schema `Course` (có trường `students`).
4. Khi học viên đăng ký khóa học:
  - Thêm ID khóa học vào `enrolledCourses` của `User`.
  - Thêm ID học viên vào `students` của `Course`.
5. Tạo route lấy danh sách học viên theo khóa học, và ngược lại.

## 9. Kết quả cần đạt:

- Hiểu rõ mô hình nhiều-nhiều.
- Biết cách cập nhật song song hai collection.
- Dùng `.populate()` cho dữ liệu chéo.

## 10. Test Case:

### Test 1: Tạo người học

json

```
{ "name": "Mai Lan", "email": "lan@example.com" }
```

## ✓ Test 2: Đăng ký khóa học

- POST /register
- Body:

json

```
{ "userId": "<userId>", "courseId": "<courseId>" }
```

## ✓ Test 3: Lấy danh sách học viên của một khóa học (có populate)

### BÀI TẬP 03: Ứng dụng Quản lý Sản phẩm và Đánh giá

## ✓ 1. Tên bài tập:

**Hệ thống Quản lý Đánh giá sản phẩm**

## 2. Mục đích:

- Thực hành mối quan hệ một-nhiều giữa `Product` và `Review`.
- Luyện dùng `populate` để hiển thị đánh giá kèm thông tin người đánh giá.

## 3. Tóm tắt nội dung:

Tạo hệ thống quản lý:

- Sản phẩm (**Product**)
- Đánh giá (**Review**)
- Người dùng (**User**)

Mỗi review liên kết với `User` và `Product`.

## 4. Yêu cầu:

- Tạo model `User`, `Product`, `Review`.
- Mỗi review chứa:
  - `user: ObjectId`
  - `product: ObjectId`
  - `rating, comment`
- API cho phép:
  - Người dùng tạo đánh giá cho sản phẩm.
  - Lấy thông tin sản phẩm kèm đánh giá và người đánh giá.

## 5. Chức năng:

- Tạo sản phẩm



- Đăng đánh giá
- Lấy thông tin sản phẩm kèm đánh giá (dùng populate)

## 6. Công nghệ:

- Node.js
- Express
- MongoDB
- Mongoose

## 7. Cấu trúc dự án:

bash

```
review-app/
├── models/
│   ├── user.model.js
│   ├── product.model.js
│   └── review.model.js
├── routes/
│   ├── product.routes.js
│   └── review.routes.js
├── app.js
└── .env
```

## 8. Các bước thực hiện:

1. Tạo các model với trường populate (ref).
2. Tạo API tạo sản phẩm, đánh giá.
3. Trong API lấy sản phẩm → `.populate('reviews').populate('reviews.user')`

## 9. Kết quả cần đạt:

- Thành thạo mô hình một-nhiều.
- Hiểu cách **populate lồng nhau** (`reviews → user`).
- Biết tổ chức schema phù hợp logic thực tế.

## 10. Test Case:

### Test 1: Tạo người dùng và sản phẩm

### Test 2: Gửi đánh giá sản phẩm

json

```
{
  "userId": "<userId>",
  "productId": "<productId>",
  "rating": 5,
  "comment": "Sản phẩm rất tốt!"
}
```

### **Test 3: Lấy chi tiết sản phẩm**

- Bao gồm tên sản phẩm, danh sách đánh giá và người đánh giá.